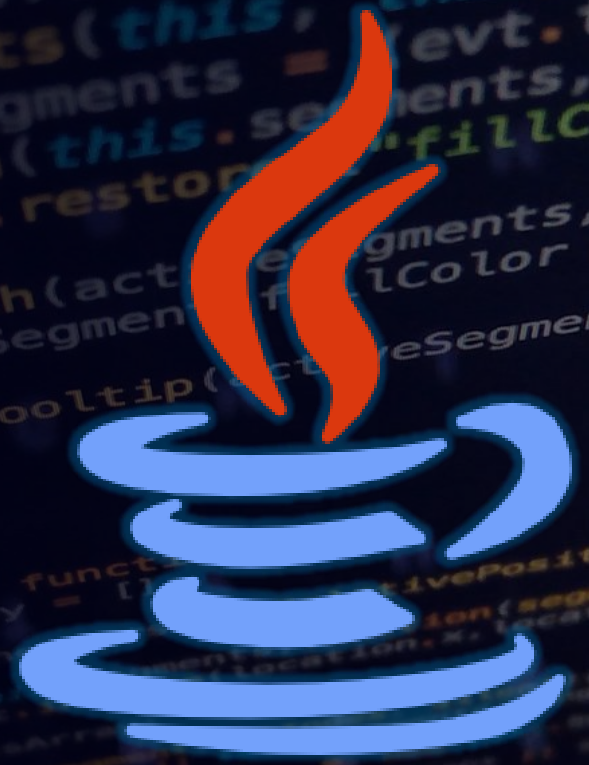
CURSO DE JAVA Y PROGRAMACIÓN ORIENTADA A OBJETOS



PROFESOR:
ANGEL MARISCURRENA

Nivel: Básico

¿POR QUÉ APRENDER JAVA EN 2026?

¿POR QUÉ APRENDER JAVA EN 2026?

Dominio del mercado laboral

Fundamento de lógica y estructura

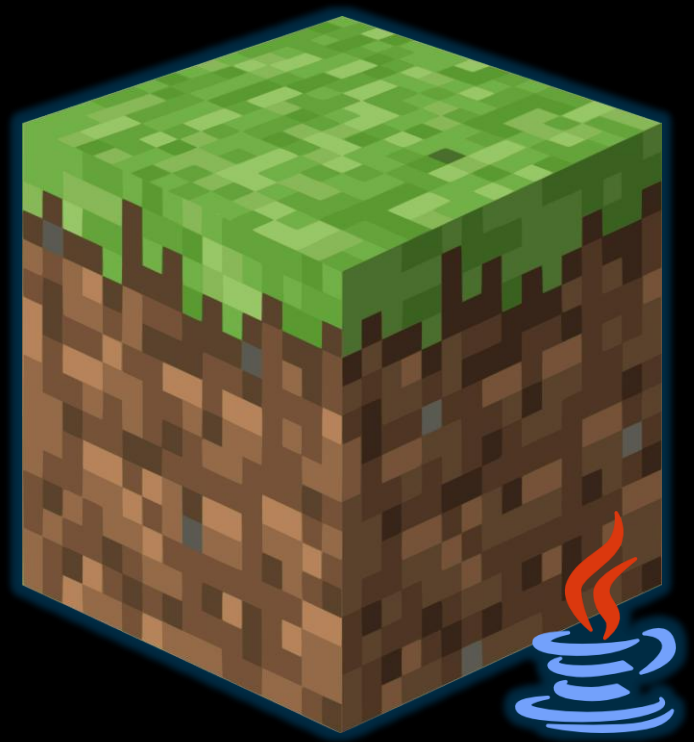
Evolución constante

El motor de Android y más

NETFLIX

amazon

Comunidad y soporte

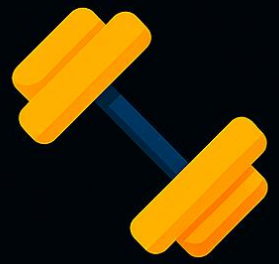


Minecraft fue desarrollado en 2009 por Markus Persson utilizando principalmente Java

¿REALMENTE ES JAVA DIFÍCIL?

No es difícil,
es “Hablador”

Verboso =
Control Total



El Gimnasio de
la Programación

El Estándar de la
Industria

COMPARACIÓN | ANALÓGIA

Python

```python
print("Hola Mundo")
```

Java

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Hola Mundo");
    }
}
```

“Un café por favor…”

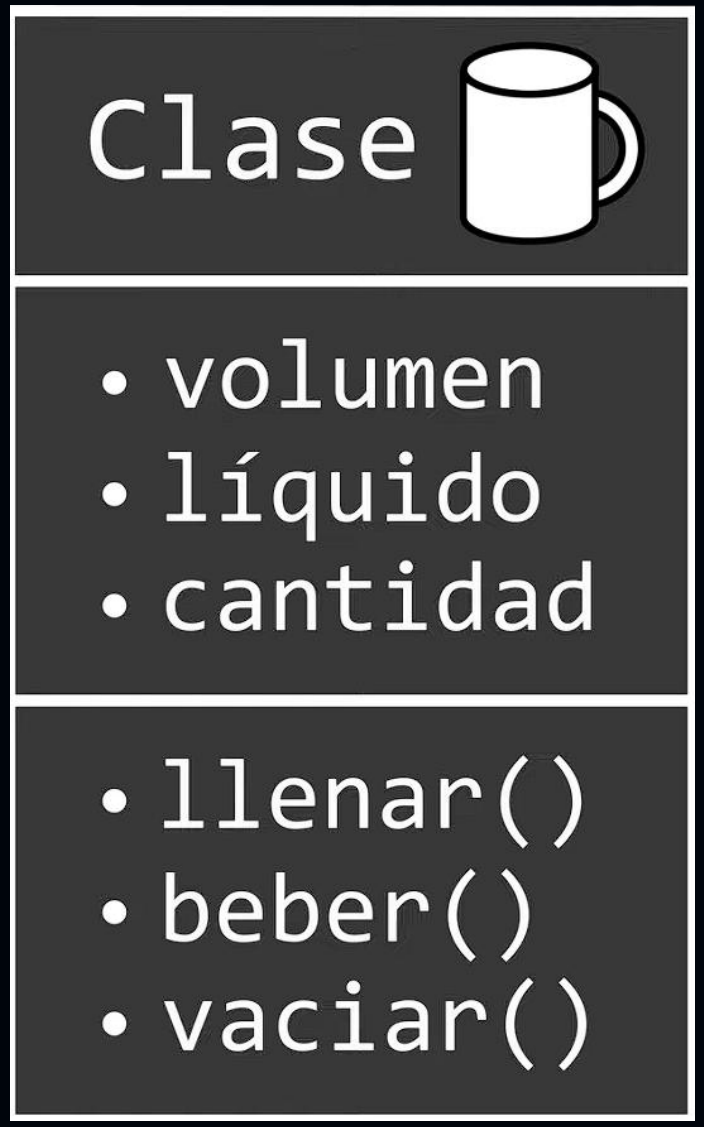
“Un café, por favor, use la máquina marca X, ponga un filtro de papel, añada 20g de grano molido fino, vierta agua a 90°C y sírvamelo en una taza de cerámica…”

{POO}

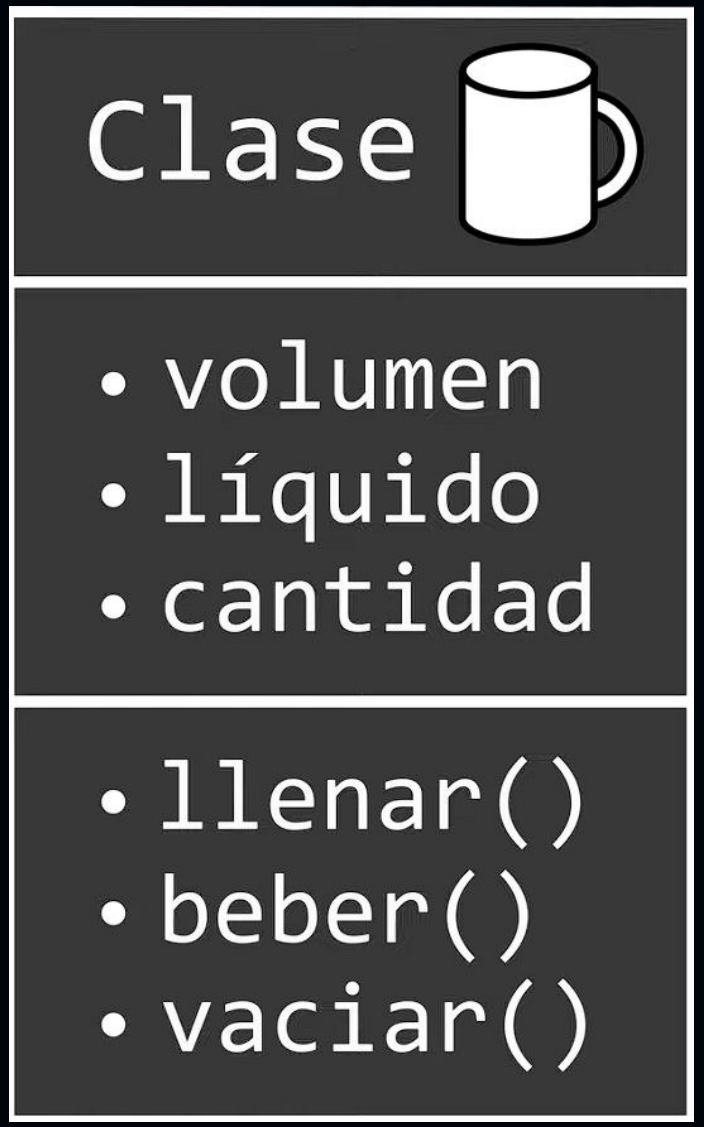
PROGRAMACIÓN ORIENTADA A OBJETOS

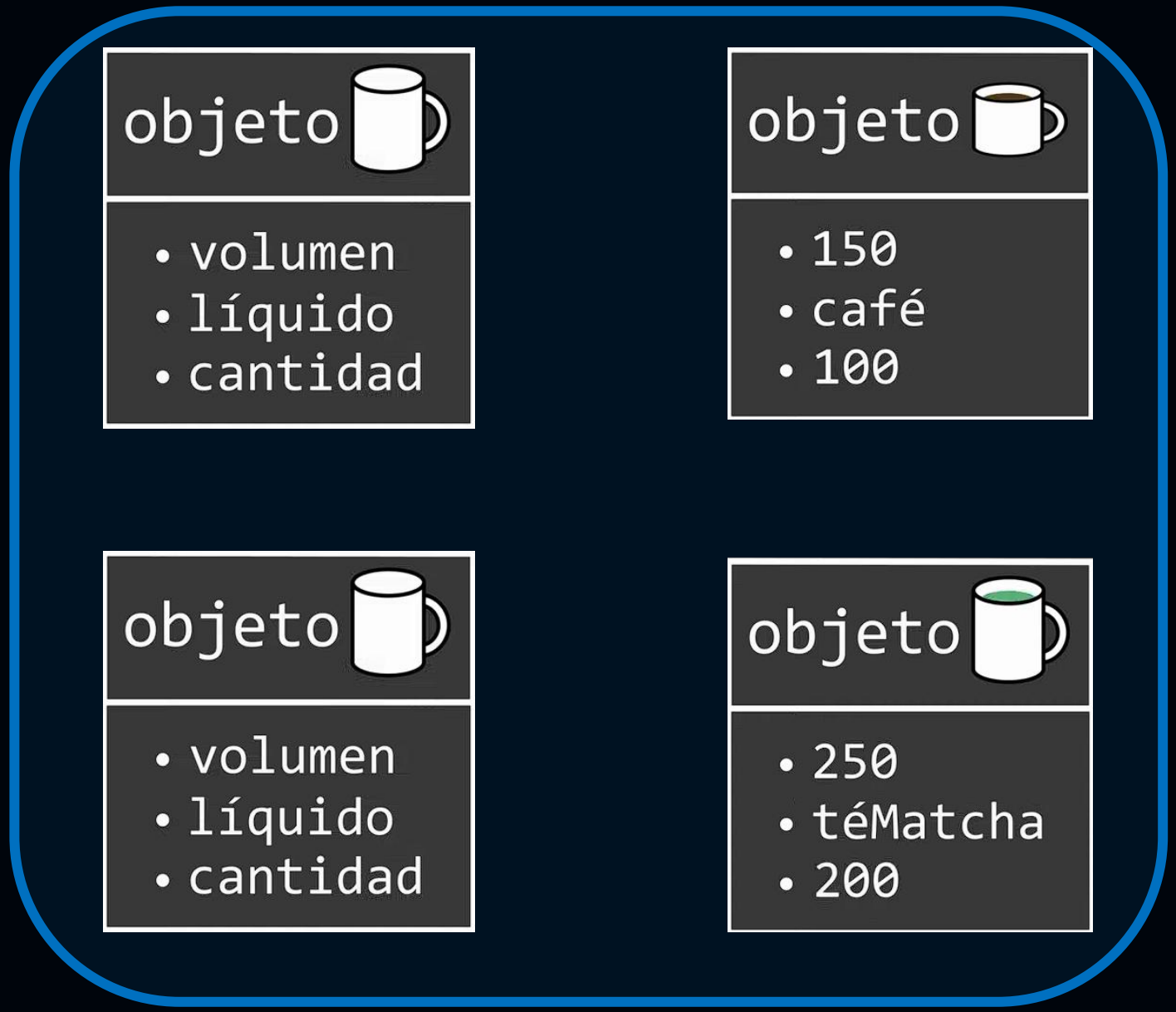
¿QUÉ ES POO?

Objeto



¿QUÉ ES POO?


Clase
• volumen
• líquido
• cantidad
• llenar()
• beber()
• vaciar()


Instanciar
objeto
• volumen
• líquido
• cantidad
objeto
• 150
• café
• 100
objeto
• volumen
• líquido
• cantidad
objeto
• 250
• téMatcha
• 200

¿QUÉ ES POO?

Clase

- volumen
- líquido
- cantidad

- llenar()
- beber()
- vaciar()

constructor (150,café,100) →

objeto

- 150
- café
- 100

Python: __init__ (150,café,100)

Java: Class Name (150,café,100)

constructor (250,téMatcha,200) →

objeto

- 250
- téMatcha
- 200

¿QUÉ ES POO?

miTaza1.cantidad

miTaza1.cantidad = 0

miTaza1.beber()

miTaza2.liquido

miTaza2.cantidad = té

miTaza2.llenar()

Ejemplo: “Un personaje de Videojuego”

Clase: Diseño del personaje (Vida, Nombre, habilidades)

Instanciar: Press Start – Aparece el personaje en pantalla

Objeto: Nuestro personaje ya corriendo por el juego.

Inicializar: Asignar 100% de vida y el nombre “Player1”.

Atributos: Su nivel de energía, su traje y su fuerza.

Métodos: El botón de saltar, correr y atacar.

Cuatro Pilares de POO

Abstracción

Encapsulamiento

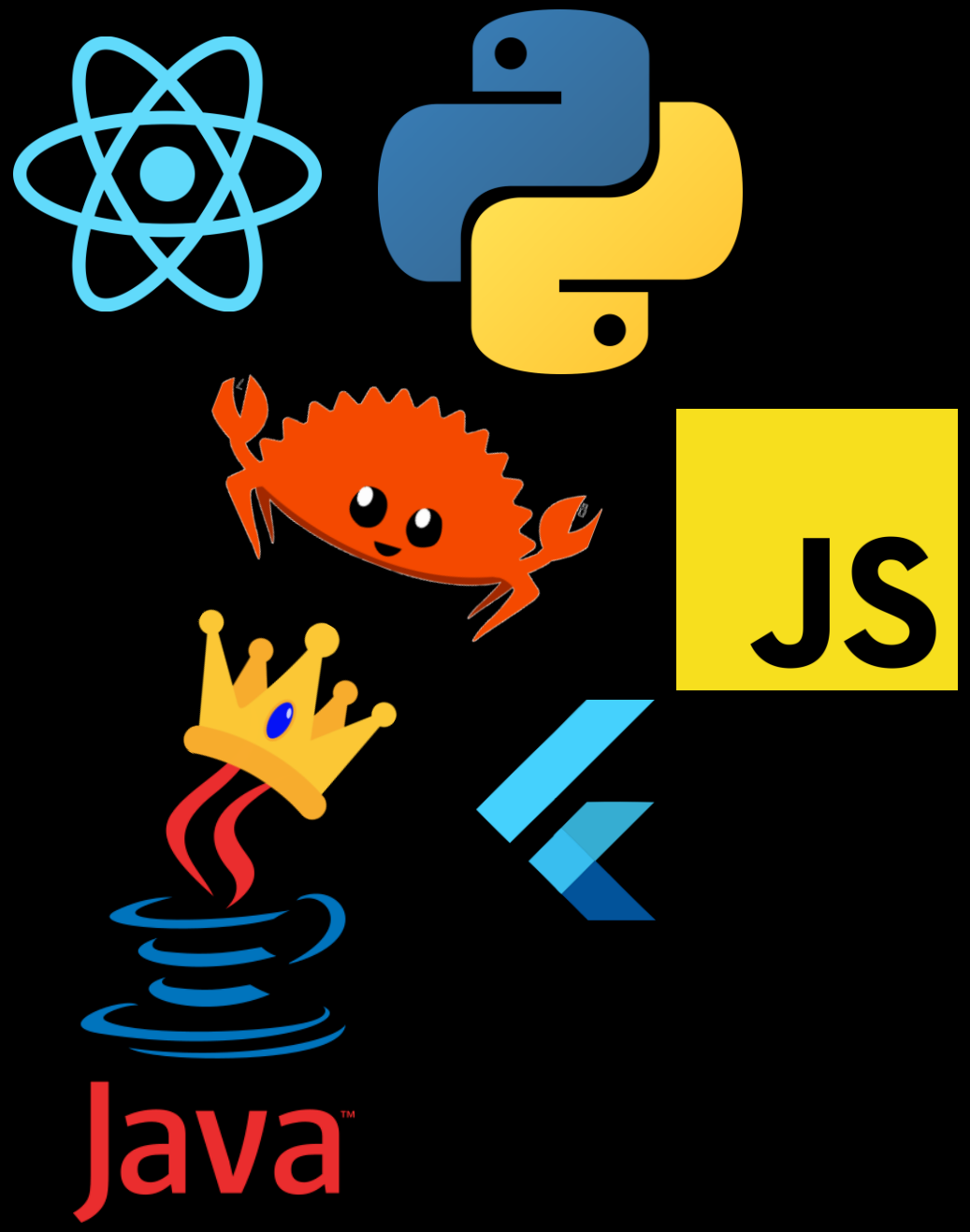
Herencia

Polimorfismo

Java y la POO

- La Programación Orientada a Objetos (POO) define el comportamiento de todos los lenguajes actuales de alto nivel.

- Java se diseño como un lenguaje orientado a objetos.



BRAIN TRAINING

---→ REALIZA UNA ANALOGÍA DE LO QUE ES UN OBJETO DE CLASE

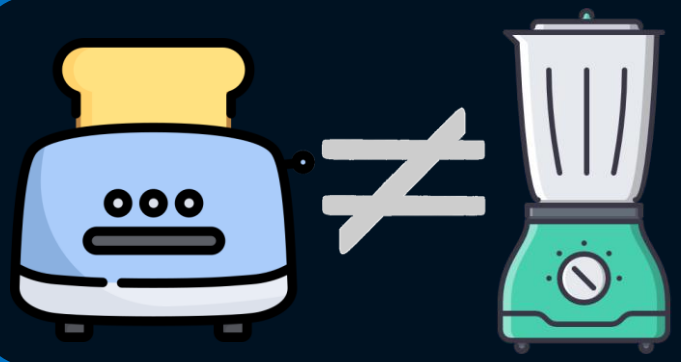
HINT: Puedes usar lo que quieras; *Un avión, un plato, una maleta, una persona*

{.java}

EL ORIGEN DE UNA REVOLUCIÓN

Historia de Java

- Año 1991



Todo se debe desarrollar una y otra vez

James Gosling

OAK

Proyecto Green

- Año 1995

Cambio de nombre

"Write Once, Run Anywhere"

Lanzamiento de la Versión 1.0

- Año 2006

OpenJDK

Java se vuelve libre

Historia de Java

2009 - Markus Persson desarrolla Minecraft

ORACLE

2010 – Oracle compra Sun Microsystems

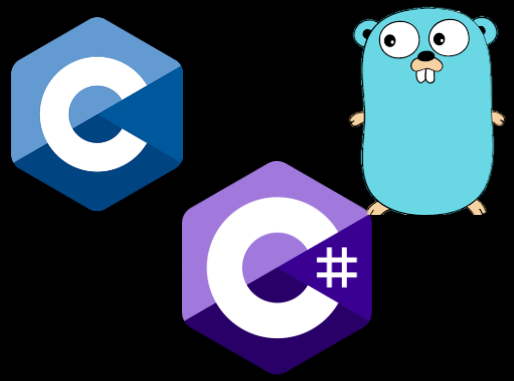
Java™

{JVM}

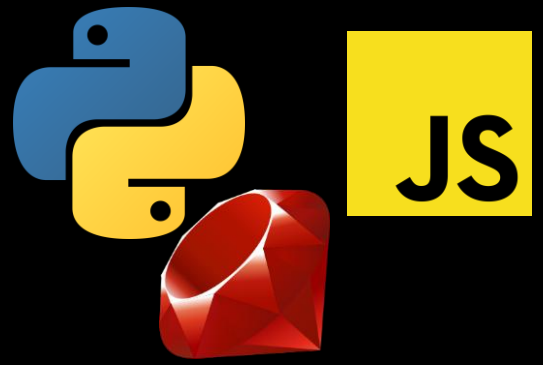
¿CÓMO JAVA LOGRA EJECUTARSE DONDE SEA?

¿ Java ?

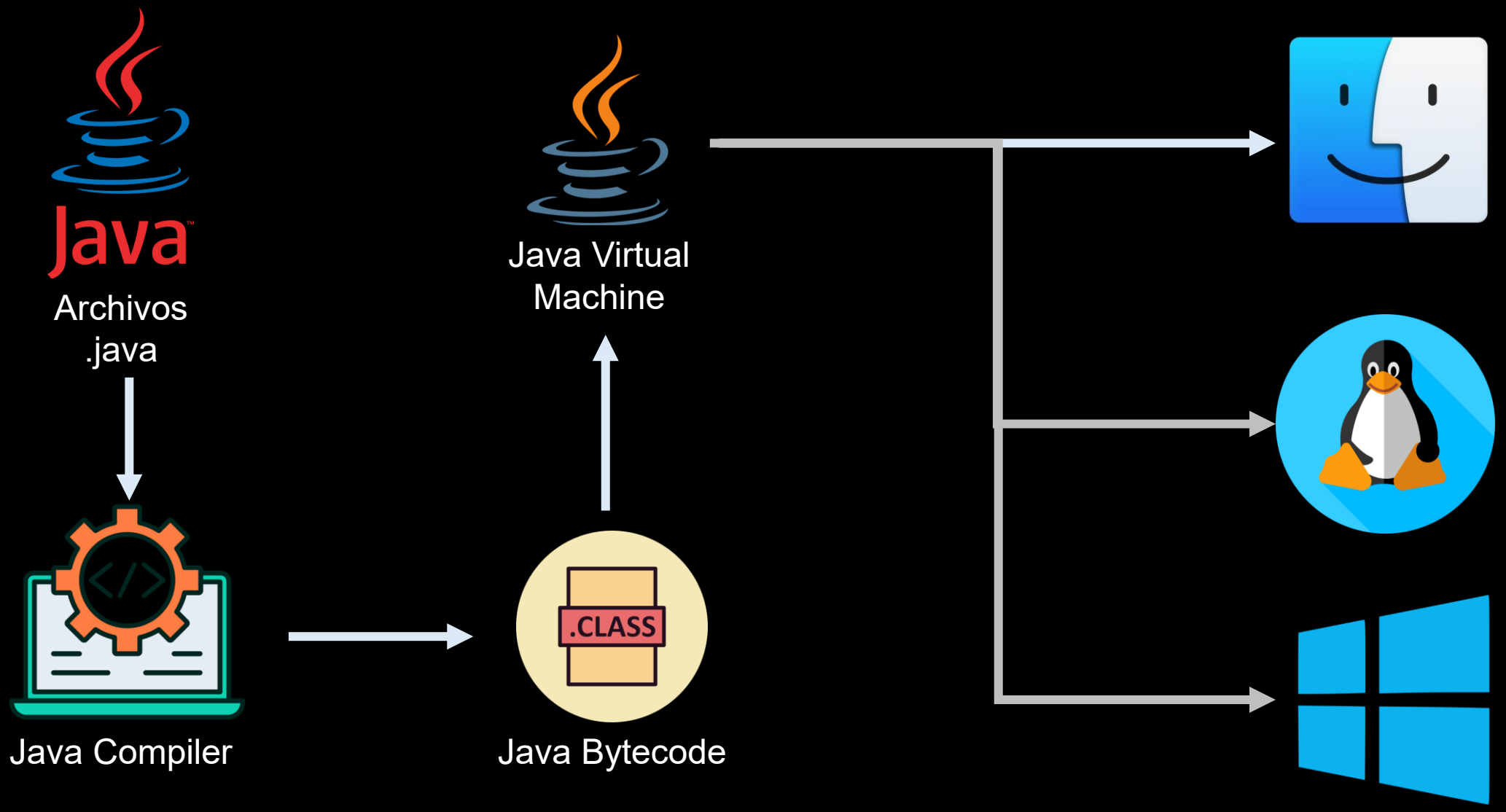
Lenguajes Compilados



Lenguajes Interpretados



¿CÓMO FUNCIONA JAVA?



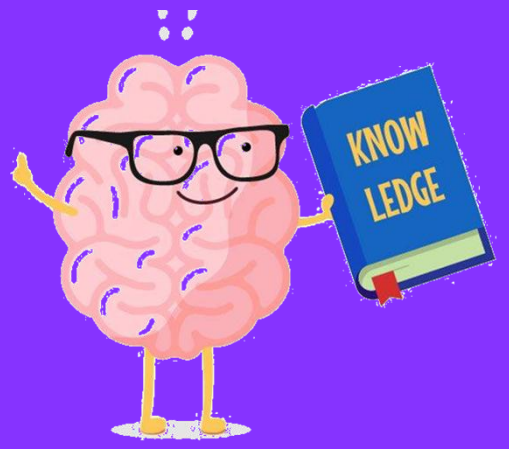
¿CÓMO FUNCIONA JAVA?

Lenguajes Copilados

Lenguajes Híbridos

Lenguajes Interpretados

BRAIN TRAINING



---→ REALIZA UNA ANALOGÍA DE LAS DIFERENCIAS ENTRE LENGUAJES COMPILADOS, INTERPRETADOS Y JAVA

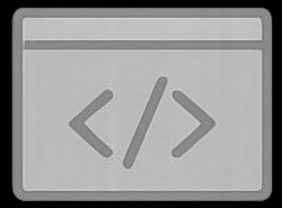
HINT: Puedes usar lo que quieras; *Un restaurante, un mueble, un disco.*

{uses}

USOS DE JAVA

USOS DE JAVA

Desarrollo de Aplicaciones Móviles (Android)



Software Empresarial

Big Data y Análisis de Datos

Internet de las Cosas (IoT)

BRAIN TRAINING

---> MENCIONA QUE TE GUSTARÍA DESARROLLAR USANDO JAVA

HINT: Es mejor soñar en grande :)

{JDK}

VERSIONES DE JAVA

VERSIONES DE JAVA

Java (*Java 8, Java 11, etc*)

- El programa para ejecutar aplicaciones Java (e.g. Minecraft)

JDK (*Java Development Kit*)

- Caja de herramientas.
- Orientada al programador.
- Escribir y crear programas.

Java SE (*Standard Edition*)

- Versión de Java libre para usar sin costo (Estudio, aprendizaje, investigación).

JRE (*Java Runtime Environment*)

- Mínimo necesario para que un programa funcione.
- Orientada al usuario.

OpenJDK (*Open Source*)

- Java de código abierto.
- Se puede distribuir libremente.
- No necesita licencia.
- Varias versiones.

Java EE (*Enterprise Edition*)

- Versión de Java empresarial.
- Incluye más herramientas.
- Tiene un costo de licencia.
- Incluye soporte.

VERSIONES DE JAVA

Java (*Java 8, Java 11, etc*)

- El programa para ejecutar aplicaciones Java (e.g. Minecraft)

JRE (*Java Runtime Environment*)

- Mínimo necesario para que un programa funcione.
- Orientada al usuario.

Java 8 →

Incluye:

- JDK (Caja de Desarrollador)
- JRE (Entorno de ejecución)
- JVM (Motor de traducción)
- API (Bibliotecas ya escritas)

JRE →

Incluye:

- JRE (Entorno)
- JVM (Motor)

Actualmente al instalar JAVA SE se instala todo por defecto.

BRAIN TRAINING

---> ¿QUÉ ES JVM Y PARA QUÉ SIRVE?

HINT: El profe dijo algo de un traductor, ¿no? :o

{JDK}

INSTALAR JAVA

INSTALAR JAVA

`java --version` — *Verifica la versión de Java instalada (Si hay alguna)*

`https://openjdk.org/version` — *Web Oficial OpenJDK (Linux, MacOS, Windows)*

`https://www.oracle.com/java` — *Web Oficial Oracle (Java SE)*

Versión para el curso

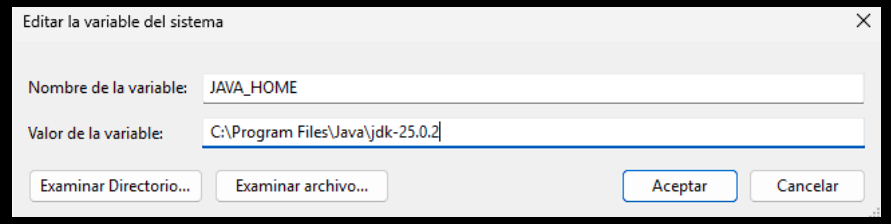
`Java 25 LTS (Long Term Support)`

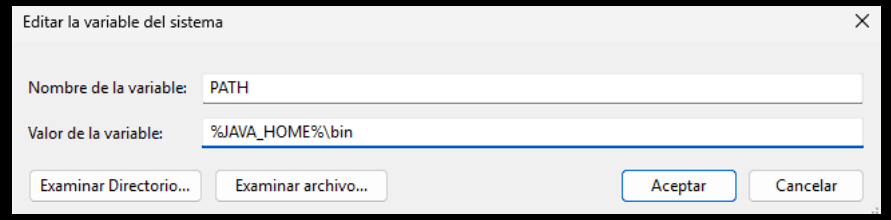
INSTALAR JAVA EN WINDOWS

1. ***Descargar el instalador Java (.msi o .exe) –*** *Se recomienda la ultima versión LTS.*

2. ***Seguir el instalador (Next) –*** *Se recomienda no cambiar la carpeta de instalación para evitar errores.*

3. ***Configura las variables de entorno –*** *Es crucial para que el SO localice el JDK y permita ejecutar las apps Java desde cualquier terminal.*
   I. *Busca "`Variables de Entorno`" en la barra de Windows.*
   II. *Crea una nueva variable de entorno llamada "`JAVA_HOME`".*
   III. *En `JAVA_HOME` asigna la ruta del `JDK`.*
   IV. *Crea una nueva variable de entorno llamada "`PATH`".*
   V. *En `PATH` asigna "`%JAVA_HOME%\bin`".*





4. *Ejecuta "`java -versión`"*

```
C:\Users\angel>java --version
java 25.0.2 2026-01-20 LTS
Java(TM) SE Runtime Environment (build 25.0.2+10-LTS-69)
Java HotSpot(TM) 64-Bit Server VM (build 25.0.2+10-LTS-69, mixed mode, sharing)
```

INSTALAR JAVA EN LINUX O MACOS

- ***Instalar software en Linux o MacOS es más sencillo.***

steve@steve-virtual-machine: ~

```
steve@steve-virtual-machine:~$ java --version
openjdk 25.0.2 2026-01-20
OpenJDK Runtime Environment (build 25.0.2+10-Ubuntu-122.04)
OpenJDK 64-Bit Server VM (build 25.0.2+10-Ubuntu-122.04, mixed mode, sharing)
steve@steve-virtual-machine:~$ sudo apt install openjdk-25-jdk
```

steve@steve-virtual-machine: ~

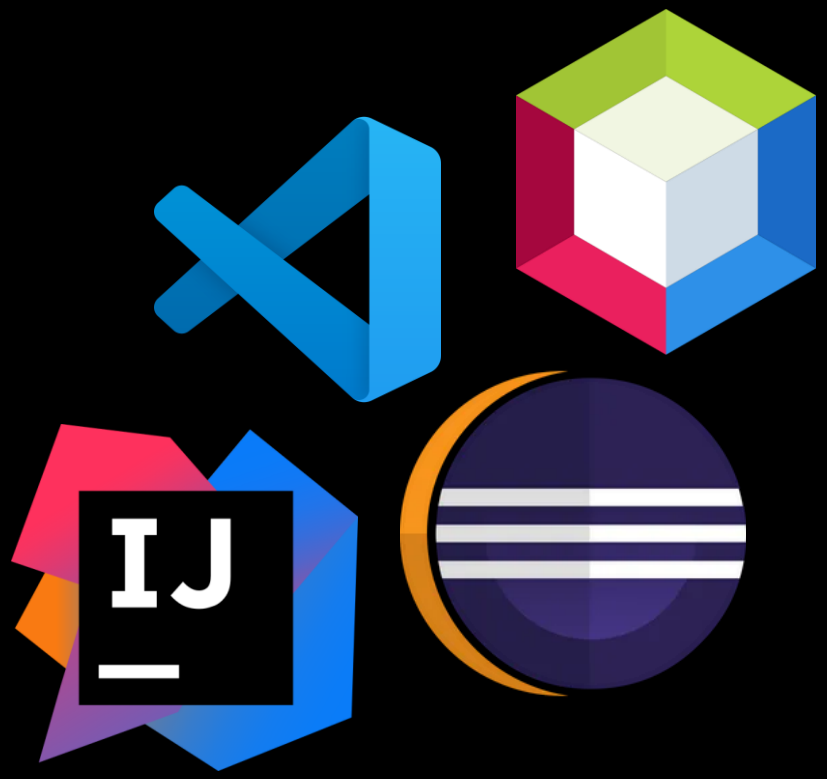
```
steve@steve-virtual-machine
---------------------------
OS: Ubuntu 22.04.5 LTS x86_64
Host: VMware Virtual Platform None
Kernel: 6.8.0-90-generic
Uptime: 2 mins
Packages: 1711 (dpkg), 11 (snap)
Shell: bash 5.1.16
Resolution: 3238x1238
DE: GNOME 42.9
WM: Mutter
WM Theme: Adwaita
Theme: Yaru-blue-dark [GTK2/3]
Icons: Yaru-blue [GTK2/3]
Terminal: gnome-terminal
CPU: AMD Ryzen 7 5700X (4) @ 3.399GH
GPU: 00:0f.0 VMware SVGA II Adapter
Memory: 1575MiB / 15946MiB
```

{IDE}

INSTALAR IDE

¿QUÉ ES UN IDE?

- **IDE** (Integrated Development Environment / Entorno de Desarrollo Integrado)
- Un editor de código diseñado para programar.



- **¿Por qué?**
  - Es más fácil que escribir en un bloc de notas.

- *Resalta colores.*
- *Tiene autocompletado.*
- *Ayuda a encontrar errores.*
- *Compila con un click.*

IDE PARA JAVA

IDE para el curso

Visual Studio Code

✅ Es muy ligero.
✅ Es rápido y sirve para todo.

❌ Necesitas instalar muchas extensiones para funcionar con Java.

Netbeans

✅ Bueno para estudiantes.
✅ Visual para aprender lo básico.

❌ Es un poco viejo en comparación de los estándares actuales.

Eclipse IDE

✅ Fue el rey por años.
✅ Es gratuito.

❌ Muy pesado.
❌ Confuso de configurar.

IntelliJ

✅ Diseñado específicamente para Java.
✅ El más inteligente y moderno del mercado.
✅ Sus desarrolladores crearon Kotlin.
✅ Expertos en Java.

INSTALAR INTELLIJ

1. ***Ve a la pagina oficial de JetBrains –*** https://www.jetbrains.com/idea/

2. ***Descarga la versión acorde a tu SO –*** *Windows.*

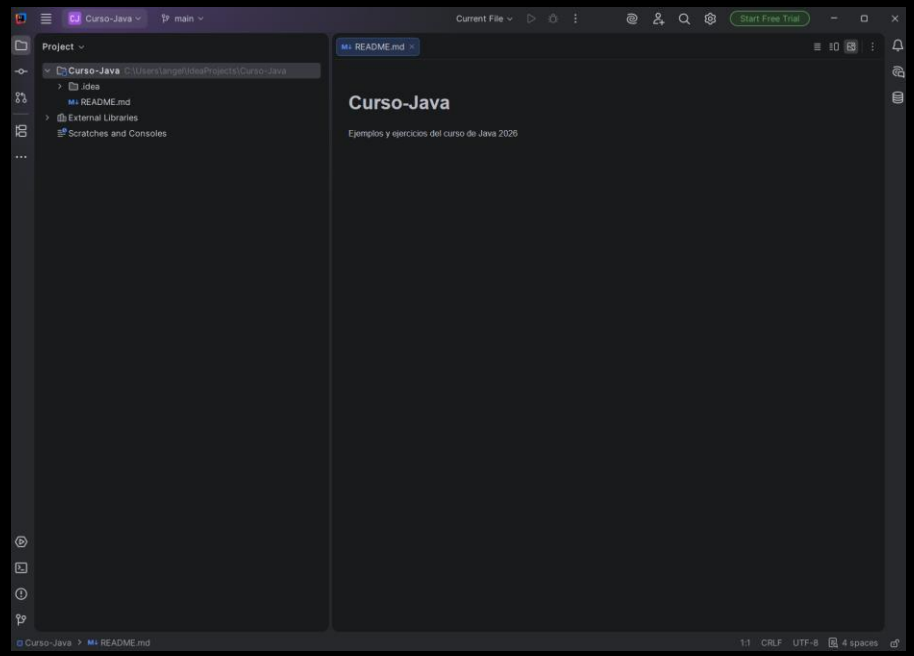
3. ***Ejecuta el archivo de instalación –*** *Cuando se complete la descarga, ejecuta el .msi.*

4. *En el instalador selecciona las casillas:*
   - ✅ Create Desktop Shortcut
   - ✅ Update PATH Variable
   - ✅ Update Context Menu
   - ✅ Create Associations (.java)

5. **Elige el tema oscuro –** Mejor para la vista.

6. **Vincular el JDK –** Es importante vincular el JDK que instalamos previamente.

7. **Opcional –** Instalar One Dark Theme



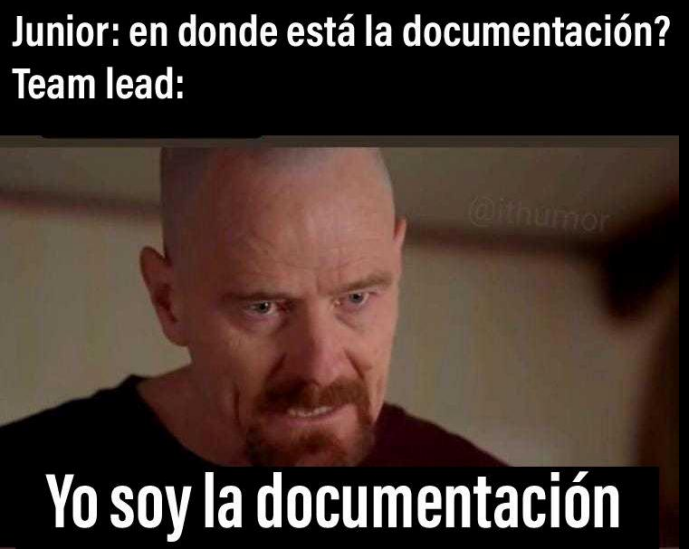
{docs}

DOCUMEN TACIÓN

DOCUMENTACIÓN DE JAVA

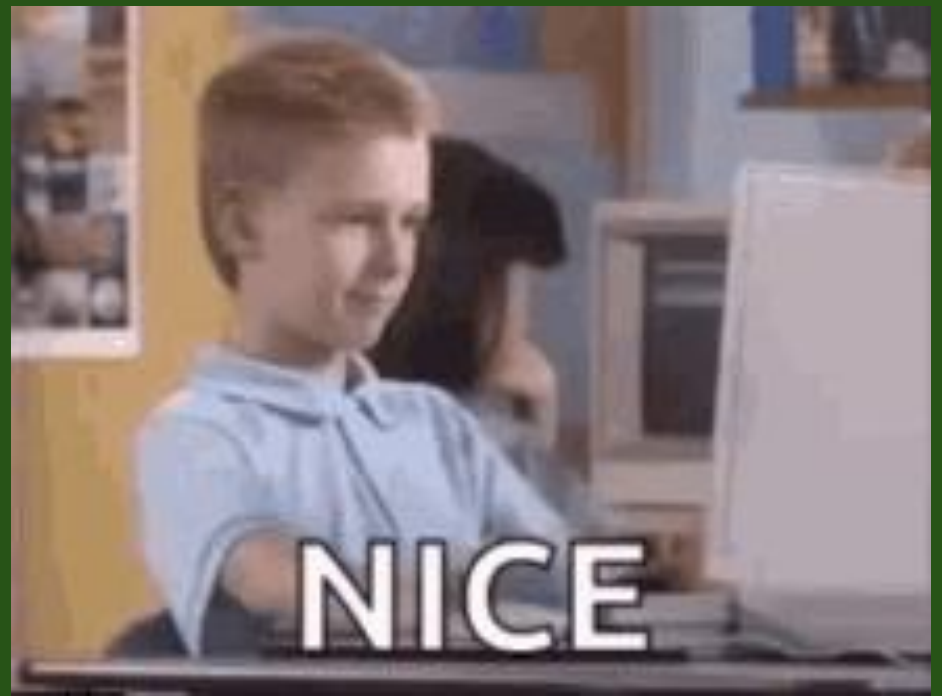
- **La documentación oficial de Java no es muy amistosa**
  – Se recomienda a los principiantes usar alternativas (Google, artículos, IA)
  https://docs.oracle.com/en/java/

- **Usar una guía de estilo** – Buenas prácticas para seguir el estándar al desarrollar en Java (Nombre de ficheros, clases, funciones, constantes, etc)
  https://google.github.io/styleguide/javaguide.html



{java}

A PROGRAMAR!



Clase 0 – Hello World

Nombre reservado:
- *Nombre reservado del método, Java siempre busca la palabra "main" para iniciar la ejecución.*

Modificador de acceso:
- *La clase es visible y se puede usar en cualquier otra parte del programa*

Definición de un objeto:
- *En Java todo debe vivir dentro de una clase.*

Nombre de la clase:
- *Por conveniencia empieza con Mayúscula y el archivo debe guardarse con el mismo nombre.*

Parámetros:
- *Permite que el programa reciba datos desde fuera (la consola) al momento de arrancar.*

```java
public class HW_Test {
    public static void main(String[] args) {
        System.out.println("Hola Mundo!\n - Primer programa en Java");
    }
}
```

static:
- *Permite que el método se ejecute sin necesidad de crear un objeto de la clase primero.*

void:
- *Indica que el método realiza una acción pero no devuelve valor al finalizar*

system:
- *Clase predefinida de Java que nos da acceso al sistema operativo*

out:
- *Flujo de salida estándar (normalmente la consola o terminal)*

println:
- *Abreviatura de "print line". Imprime el texto y salta a la siguiente línea.*

String:
- *El texto entre comillas es un String (cadena de caracteres)*

- *Programa Básico*
- *Verificar el entorno de desarrollo.*
- *Introducir la sintaxis básica.*

Clase 0 – Comentarios y punto y coma

- **Punto y Coma (;)**
  - Terminador de sentencias.
  - Le dice a Java: "He terminado de darte esta orden específica, ahora puedes pasar a la siguiente".

```
ı Java");
```

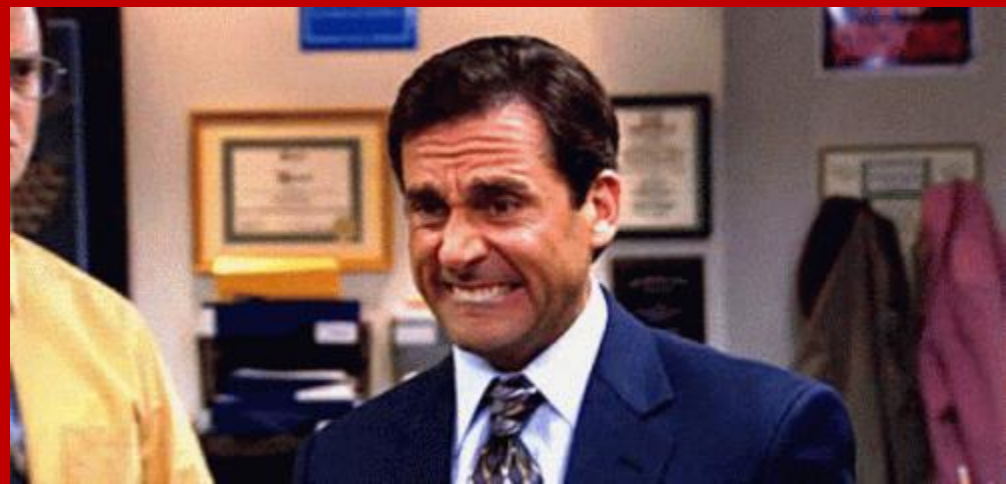
- **Comentarios**
  - Memoria a largo plazo.
  - Trabajo en equipo.
  - Depuración

```
// Ejemplo de un comentario simple

/*
Ejemplo
de un comentario
en varias lineas
de texto
*/
```

{homework}

TAREA



TAREA

1. Imprime un mensaje que diga tu nombre en lugar de "¡Hola Mundo!".
2. Imprime dos líneas: "Hola" y luego "Mundo" con un solo println.
3. Añade un comentario sobre lo que hace cada línea del programa.
4. Crea un comentario en varias líneas.
5. Imprime tu edad, tu color favorito y tu ciudad.
6. Explora los diferentes System.XXX.println(); más allá de "out".
7. Utiliza varios println para imprimir una frase.
8. Imprime un diseño ASCII (por ejemplo, una cara feliz usando símbolos).
9. Intenta ejecutar el programa sin el método main y observa el error.
10. Intenta cambiar el nombre del archivo a uno diferente del de la clase y compílalo. ¿Qué pasa?